# AI DevCoach — GitHub 프로젝트 분석 기능 상세 화면설계서

화면설계서 v1.0 | 기능 ID: FR-05 | 연계: FR-03 학습계획, FR-04 학습진도, FR-06 AI 코드 리뷰, FR-08 학습성과 분석

## 1. 기능 개요

### 1.1 개발 목적

학습자가 GitHub에 등록한 프로젝트 저장소를 AI DevCoach에 연결하면, 시스템이 프로젝트 구조, 기술 스택, 코드 구현 현황, README, Commit 이력, 학습과제 수행 여부를 분석하고 학습자의 개발 역량과 개선사항을 제공하는 기능입니다.

단순한 GitHub 저장소 조회가 아니라 다음 세 가지를 핵심으로 설계합니다.

1. 프로젝트 분석: 저장소 구조, 주요 기능, 기술 스택, 문서화 수준 파악
2. 학습 성과 검증: 개인별 학습계획의 과제 요구사항과 실제 GitHub 결과물 비교
3. AI 개선 제안: 코드 품질, 미구현 기능, 보완할 기술과 다음 학습 방향 추천

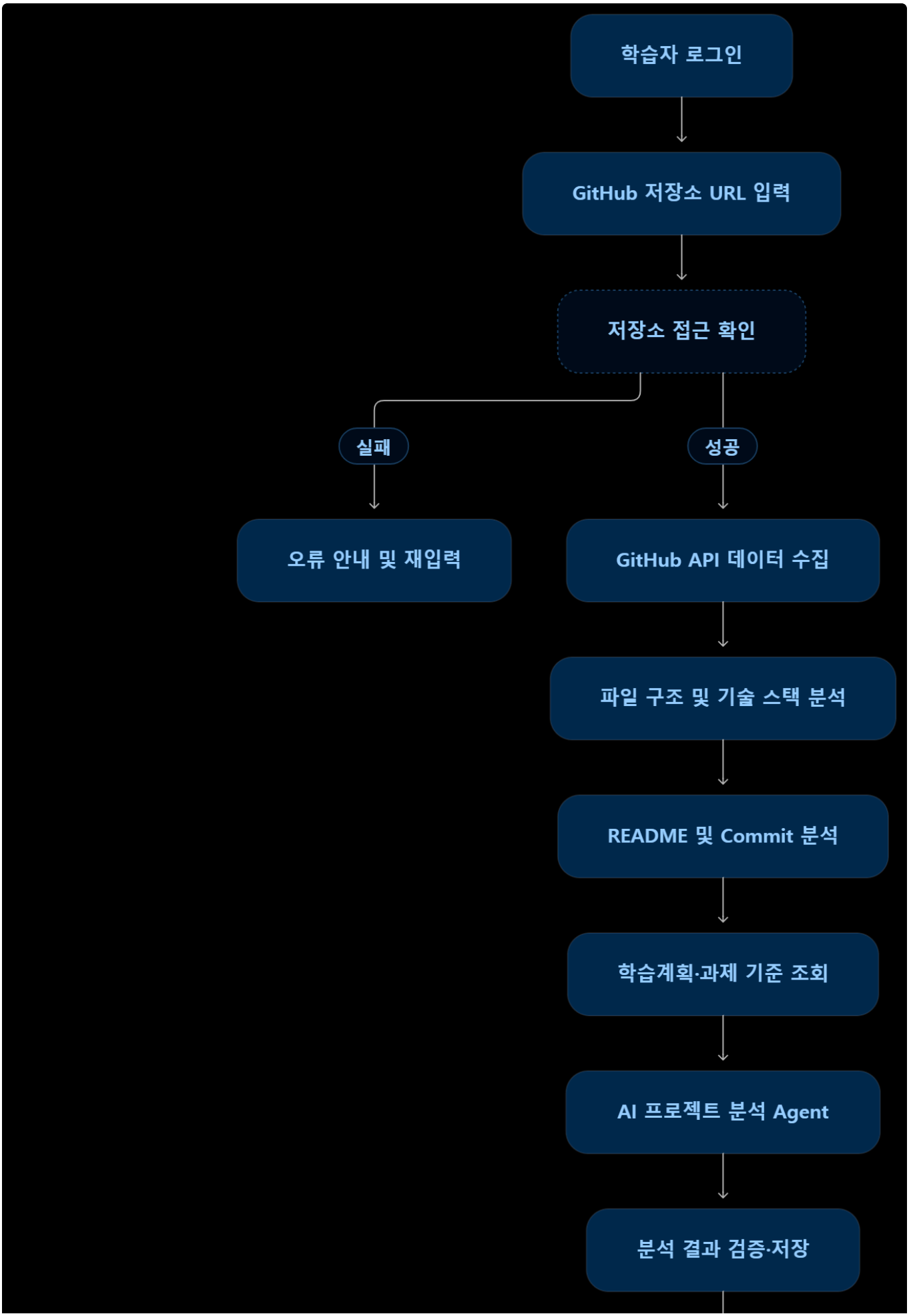
### 1.2 기능 요구사항

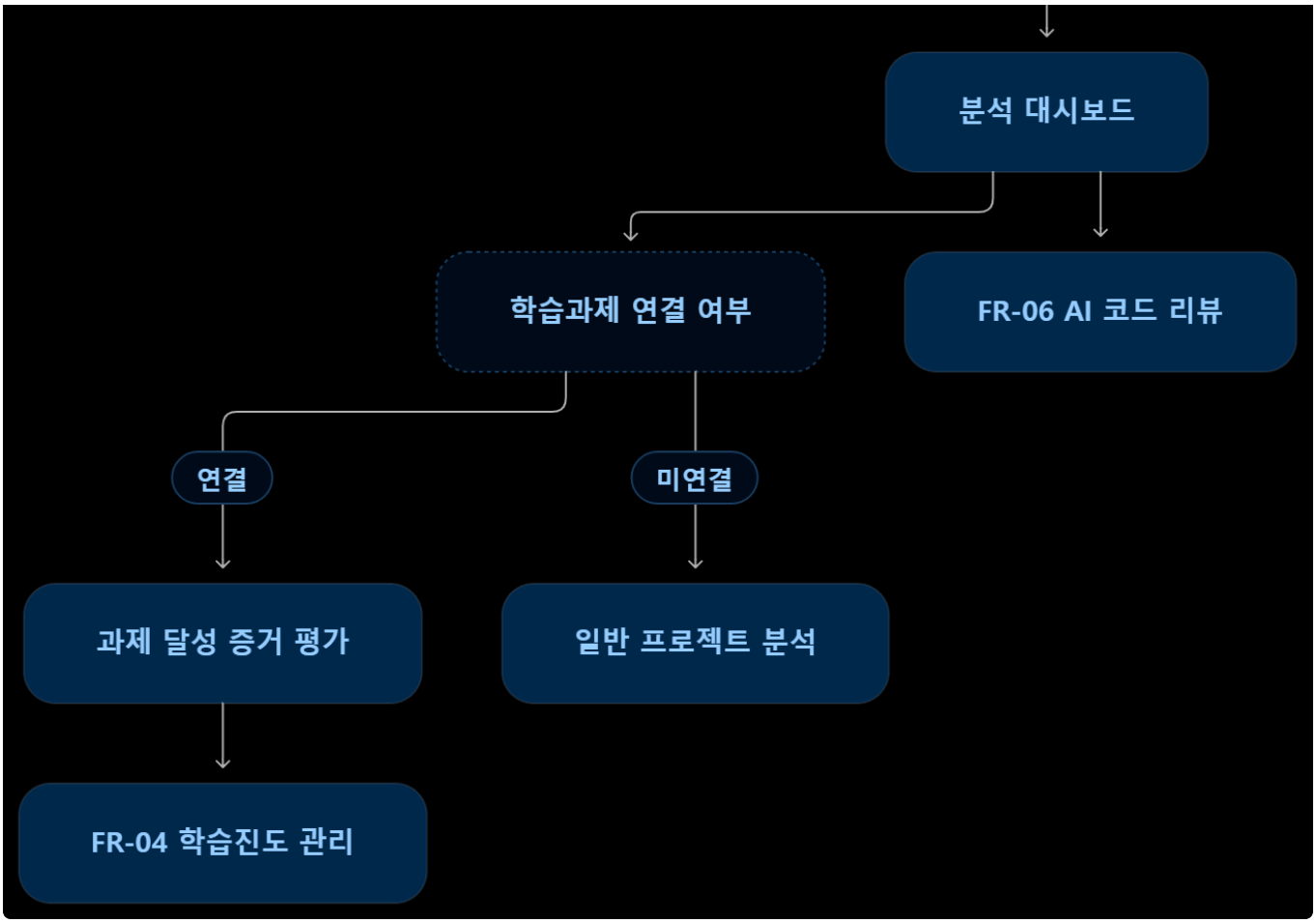
| 기능 ID | 기능명 | 주요 기능 | 우선순위 |
|---|---|---|---|
| GH-01 | GitHub 저장소 등록 | URL 입력 및 저장소 연결 | 필수 |
| GH-02 | 저장소 정보 조회 | 이름, 설명, Branch, 언어, 최근 Commit | 필수 |
| GH-03 | 프로젝트 구조 분석 | 폴더·파일 구성 및 기술 스택 분석 | 필수 |
| GH-04 | README 분석 | 설치·실행·기능·구조 문서 평가 | 필수 |
| GH-05 | Commit 활동 분석 | Commit 수, 시점, 변경 이력 | 필수 |
| GH-06 | AI 프로젝트 분석 | 구현 현황·문제점·개선사항 제안 | 필수 |
| GH-07 | 학습과제 연계 | 제출 결과물과 완료 기준 비교 | 필수 |
| GH-08 | 분석 결과 저장 | 분석 이력 및 재분석 | 필수 |
| GH-09 | AI 코드 상세 리뷰 | 파일별 코드 개선 제안 | FR-06 연계 |
| GH-10 | Pull Request 분석 | PR 및 협업 활동 평가 | 확장 |

### 1.3 화면 구성

| 화면 ID | 화면명 | 주요 기능 |
|---|---|---|
| UI-21 | GitHub 프로젝트 등록 | 저장소 URL 입력 및 연결 |
| UI-22 | 프로젝트 분석 대시보드 | 종합 분석 결과와 기술 스택 |
| UI-23 | 프로젝트 구조 분석 | 폴더·파일 구조 및 구성 적정성 |
| UI-24 | README·문서 분석 | README 충실도와 보완사항 |
| UI-25 | Commit 활동 분석 | Commit 이력과 개발 활동 |
| UI-26 | AI 프로젝트 진단 | 강점·약점·개선 우선순위 |
| UI-27 | 학습과제 연계 평가 | 학습 목표 대비 결과물 검증 |
| UI-28 | 분석 이력 | 이전 분석과 최신 분석 비교 |

## 2. 전체 업무 프로세스


학습자 로그인
GitHub 저장소 URL 입력
저장소 접근 확인
실패
성공
오류 안내 및 재입력
GitHub API 데이터 수집
파일 구조 및 기술 스택 분석
README 및 Commit 분석
학습계획·과제 기준 조회
AI 프로젝트 분석 Agent
분석 결과 검증·저장




설계 원칙: GitHub API에서 직접 확인할 수 있는 사실과 AI가 해석한 평가를 구분하여 표시합니다. Commit 수가 많다는 이유만으로 코드 품질이나 학습 역량이 높다고 판단하지 않습니다.

# 3. UI-21 GitHub 프로젝트 등록 화면

## 3.1 화면 목적

학습자가 분석할 GitHub 저장소를 지정하고, 분석할 Branch 및 연계 학습과제를 선택합니다.

## 3.2 화면 시안

아래는 입력값을 변경하고 샘플 분석 결과를 확인할 수 있는 인터랙티브 화면입니다. 실제 GitHub API 및 Neon DB에는 연결되어 있지 않습니다.




### 3.3 입력 항목 명세

| 항목 ID | 항목명 | 필수 | 처리 규칙 |
|---|---|---|---|
| GH-I01 | Repository URL | Y | `github.com/{owner}/{repo}` 형식 |
| GH-I02 | Branch | Y | 저장소 기본 Branch 자동 설정, 변경 가능 |
| GH-I03 | 연계 학습과제 | N | 현재 학습계획의 과제 선택 |
| GH-I04 | 분석 범위 | Y | 메타데이터·파일·README·Commit |
| GH-I05 | 분석 실행 | Y | GitHub 접근 확인 후 분석 작업 생성 |

실제 서비스에서는 URL 형식 검증뿐 아니라 GitHub API를 통한 저장소 존재 여부, 접근 권한, Branch 존재 여부를 확인해야 합니다.

## 4. UI-22 프로젝트 분석 대시보드

### 4.1 화면 구성

| 영역 | 표시 항목 | 데이터 출처 |
|---|---|---|
| 프로젝트 정보 | 저장소명, 설명, Branch, URL | GitHub API |
| 기술 스택 | 주요 언어, 프레임워크, 패키지 | 파일·설정 분석 |

| 영역 | 표시 항목 | 데이터 출처 |
|---|---|---|
| 프로젝트 구성 | 디렉터리, 주요 파일 | Git Tree API |
| 문서화 | README 항목별 충족 여부 | README 분석 |
| 개발 활동 | Commit 수, 최근 변경 | GitHub API |
| AI 종합 분석 | 구현 현황, 문제점, 개선사항 | AI Agent |
| 학습 연계 | 과제별 목표 달성 증거 | FR-03, FR-04 |

### 4.2 분석 결과 표시 원칙

프로젝트 분석 결과는 세 가지 상태로 구분합니다.

| 상태 | 의미 | 예시 |
|---|---|---|
| 확인됨 | 실제 저장소에서 증거 확보 | `requirements.txt` 존재 |
| 보완 필요 | 기준에 미달하는 증거 확인 | README에 실행 방법 누락 |
| 확인 불가 | 수집 범위나 접근 제한으로 판단 불가 | 외부 서비스 배포 상태 |

확인 불가를 미구현으로 단정하지 않는 것이 중요합니다.

## 5. UI-23 프로젝트 구조 분석

### 5.1 분석 항목

| 평가 항목 | 분석 방법 | 결과 예시 |
|---|---|---|
| 폴더 구조 | 디렉터리 트리 확인 | frontend/backend 분리 |
| 실행 진입점 | `main.py`, `app.py` 등 확인 | FastAPI 앱 존재 |
| 패키지 관리 | `requirements.txt`, `pyproject.toml` 확인 | 의존성 명시 |
| 환경 설정 | `.env.example` 등 확인 | 설정 안내 존재 |
| DB 구성 | SQLAlchemy 모델·설정 확인 | PostgreSQL 연동 코드 |
| API 구성 | Router·Endpoint 탐색 | 인증·과제 API |
| 테스트 구성 | 테스트 파일 및 실행 설정 확인 | pytest 파일 존재 |
| 배포 구성 | 배포 설정 파일 확인 | Vercel 설정 존재 |

프로젝트의 파일명은 다양하므로 특정 파일명만으로 기능의 존재 여부를 판정하지 않습니다. Python의 `pyproject.toml`처럼 대체 구성이 있을 수 있기 때문입니다.

### 5.2 파일 상세 화면

파일을 선택하면 다음 정보를 표시합니다.

- 파일 경로 및 파일 크기
- 마지막 변경 Commit
- 파일 역할에 대한 AI 설명
- 주요 클래스·함수
- 발견된 문제점
- FR-06 AI 코드 리뷰 실행 기능

MVP에서는 전체 파일을 무조건 LLM에 전달하지 않고, 파일 목록 → 중요 파일 선택 → 필요한 내용만 분석하는 구조를 권장합니다.

## 6. UI-24 README 분석 화면

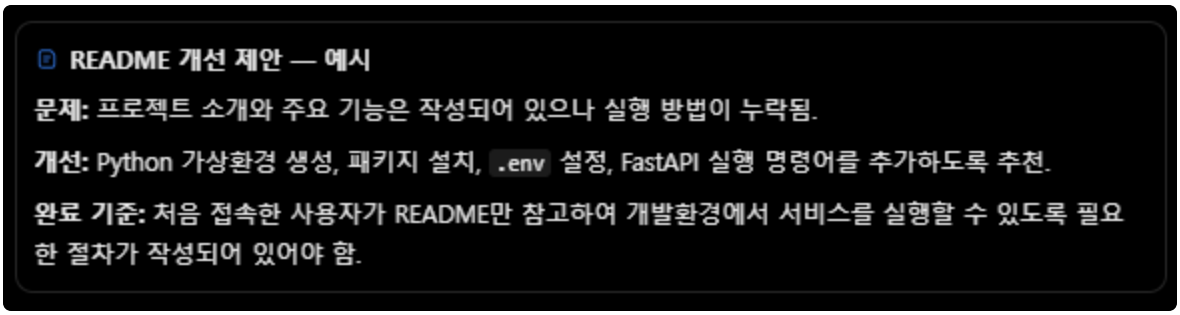
### 6.1 README 평가 기준

| 평가 항목 | 배점 | 확인 기준 |
|---|---|---|
| 프로젝트 개요 | 15 | 목적·대상 사용자·문제 정의 |
| 주요 기능 | 20 | 구현 기능 및 사용 흐름 |
| 기술 스택 | 15 | 언어·프레임워크·DB |
| 설치·실행 방법 | 20 | 설치 명령어와 실행 절차 |
| 환경변수 안내 | 10 | `.env.example` 등 설정 설명 |
| API·사용 예시 | 10 | 사용 방법·호출 예시 |
| 테스트·제약사항 | 10 | 검증 방법 및 알려진 제한 |
| 합계 | 100 | |

점수는 프로젝트 내부의 운영용 평가 기준이며, 실제 소프트웨어 품질을 공인하는 지표가 아닙니다.

### 6.2 AI 개선 제안 예시

README 개선 제안 — 예시




# 7. UI-25 Commit 활동 분석 화면

## 7.1 분석 지표

| 지표 | 분석 내용 |
|---|---|
| 전체 Commit 수 | 지정 기간 및 Branch 기준 |
| 최근 Commit | 변경일, 메시지, 작성자 |
| 주차별 Commit | 주간 개발 활동 추세 |
| 변경 파일 | 과제와 관련된 파일 변경 |
| Commit 메시지 | 변경 목적의 명확성 |
| 협업 활동 | PR, 리뷰 등 확인 가능한 활동 |

Commit 수는 학습 참여를 파악하는 보조 지표입니다. Commit 수가 많다고 구현 완성도가 높다고 평가하지 않습니다.

또한 여러 명이 함께 사용하는 저장소에서는 학습자 본인의 기여와 전체 프로젝트 활동을 구분해야 합니다.

# 8. UI-26 AI 프로젝트 진단 화면

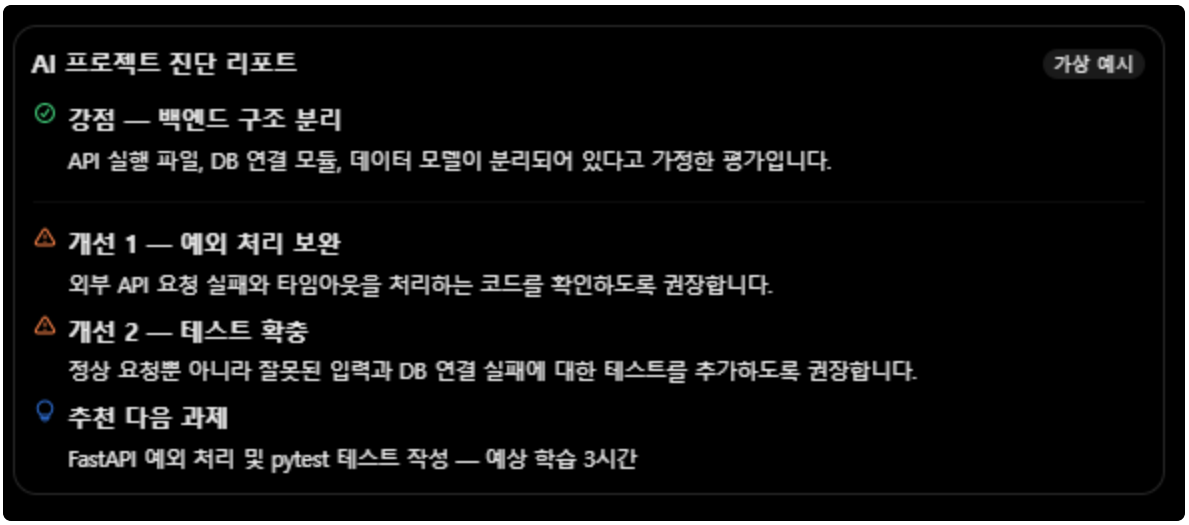
## 8.1 AI 분석 결과 구성

| 구분 | 분석 내용 |
|---|---|
| 프로젝트 요약 | 프로젝트 목적 및 구현 기술 |
| 구현된 기능 | 소스코드에서 확인 가능한 기능 |
| 미확인 기능 | 검증 증거가 부족한 기능 |
| 구조 개선사항 | 모듈 분리, 의존성, 중복 코드 |
| 품질 개선사항 | 예외 처리, 검증, 테스트 |
| 문서 개선사항 | README, 설치·실행 안내 |
| 추천 학습 | 부족한 역량과 관련 학습 과제 |

| 구분 | 분석 내용 |
| --- | --- |
| 우선순위 | 중요도와 예상 개선 효과 |

## 8.2 분석 결과 예시

AI 프로젝트 진단 리포트




## 8.3 AI 평가 근거

AI가 문제점을 제시할 때는 가능하면 다음 근거를 함께 저장합니다.

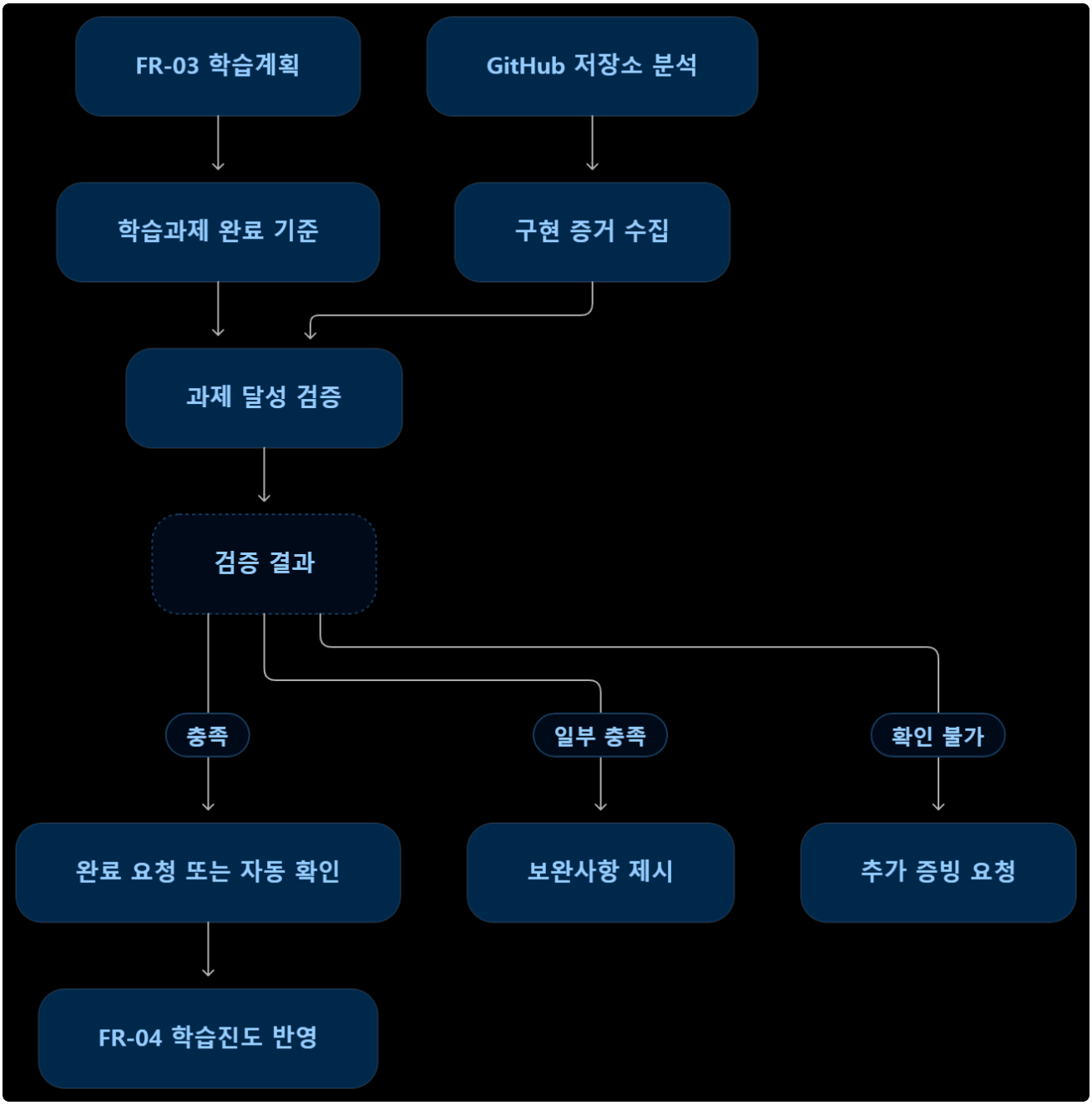
```
{
  "finding_id": "F-001",
  "category": "error_handling",
  "severity": "medium",
  "description": "API 예외 처리 확인 필요",
  "evidence": [
    {
      "file": "backend/main.py",
      "line_start": 25,
      "line_end": 32
    }
  ],
  "recommendation": "외부 API 오류 및 타임아웃 처리 추가",
  "verification_status": "needs_review"
}
```

라인 번호는 분석 대상 Commit SHA와 함께 기록해야 하며, 실제로 수집한 파일에 존재하는 범위만 인용해야 합니다.

# 9. UI-27 학습과제 연계 평가 화면

이 기능이 AI DevCoach의 일반 GitHub 분석 도구와 차별화되는 핵심입니다.

## 9.1 과제 연계 구조




## 9.2 과제 평가 예시

| 학습과제 | 완료 기준 | GitHub 확인 증거 | 평가 |
|---|---|---|---|
| GitHub 활용 | 저장소 및 README 작성 | 저장소·README 존재 | 충족 가능 |
| FastAPI 개발 | GET·POST API 구현 | 라우터 코드 확인 | 추가 테스트 필요 |
| DB 연동 | PostgreSQL CRUD 구현 | DB 모델·CRUD 코드 | 실행 검증 필요 |
| AI Agent 개발 | LLM과 Tool Calling 연동 | API 호출 및 도구 정의 | 동작 검증 필요 |

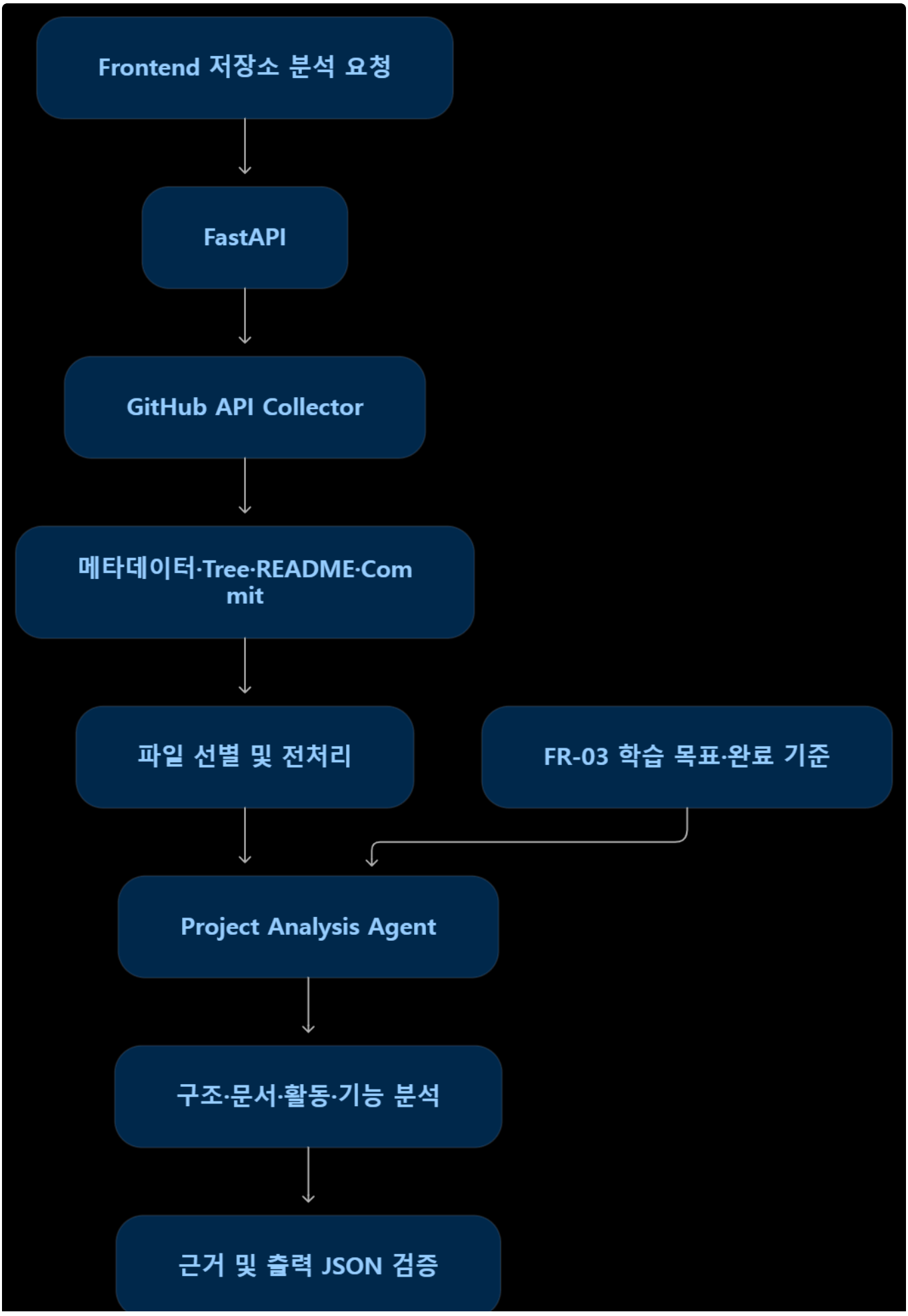
실제 동작 여부는 정적 코드 분석만으로 충분히 확인하기 어렵습니다. 가능한 경우 테스트 실행 결과, 실행 로그 또는 학습자 제출 증빙을 함께 활용합니다.

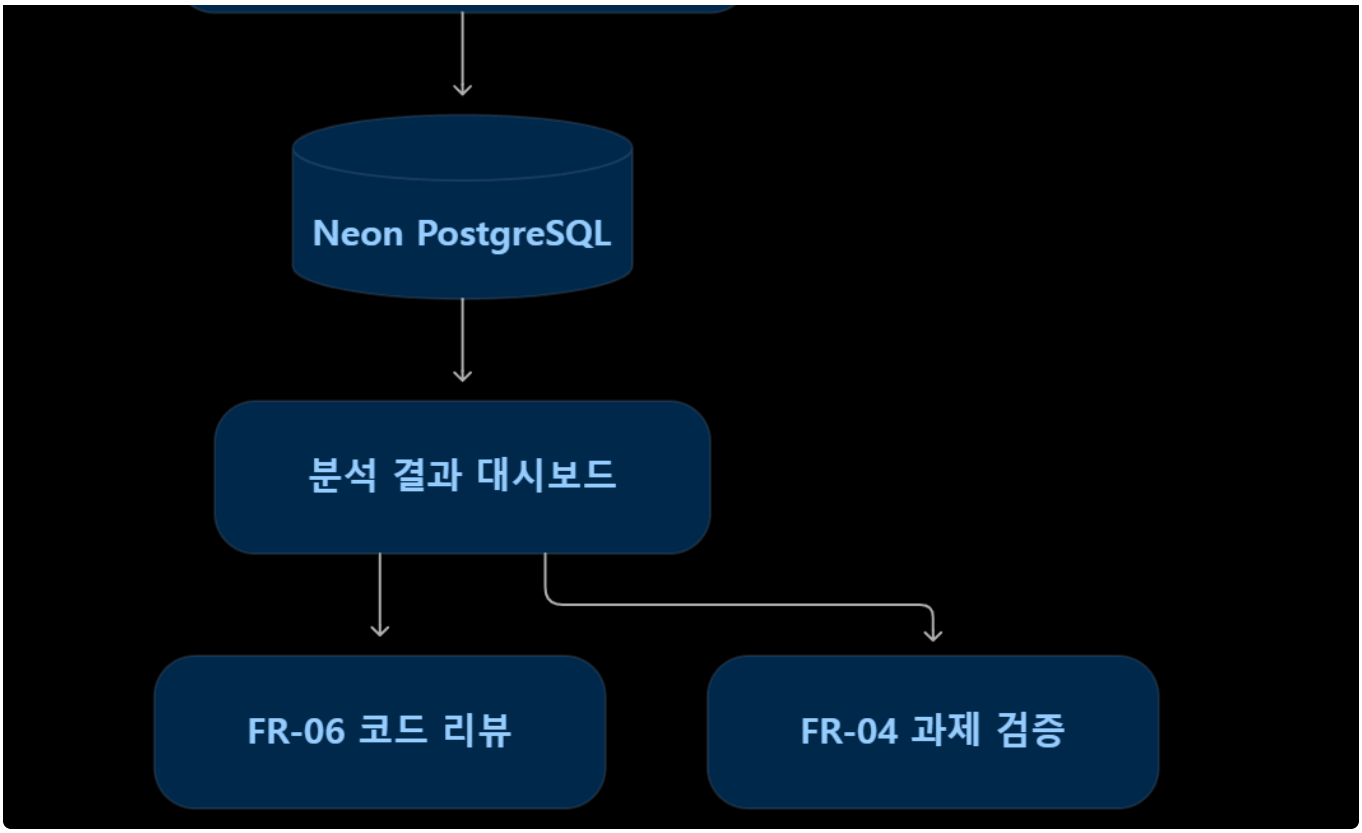
### 9.3 학습 진도 반영 정책

GitHub 분석 결과는 `task_verifications`에 저장하고, 과제 상태를 자동으로 변경하기 전 완료 기준을 확인합니다.

권장 상태는 `미검증 → 분석 중 → 충족 / 보완 필요 / 확인 불가` 입니다. 학습자가 재제출하면 새로운 분석 이력을 생성하고 이전 결과를 보존합니다.

## 10. GitHub 프로젝트 분석 Agent 아키텍처


Frontend 저장소 분석 요청
FastAPI
GitHub API Collector
메타데이터·Tree·README·Commit
파일 선별 및 전처리
FR-03 학습 목표·완료 기준
Project Analysis Agent
구조·문서·활동·기능 분석
근거 및 출력 JSON 검증




## 10.1 Agent Tool 명세

| Tool | 역할 |
|---|---|
| `get_repository_info` | 저장소 메타데이터 조회 |
| `get_repository_tree` | 디렉터리·파일 목록 |
| `get_file_content` | 선택 파일 내용 조회 |
| `get_readme` | README 조회 |
| `get_commit_history` | Commit 이력 조회 |
| `detect_tech_stack` | 언어·프레임워크 탐지 |
| `get_learning_task_criteria` | 학습과제 완료 기준 조회 |
| `analyze_project_structure` | 프로젝트 구조 평가 |
| `evaluate_project_requirements` | 과제 요구사항 대비 구현 분석 |
| `save_project_analysis` | 분석 결과 저장 |

## 10.2 GitHub API 사용

공식 문서: GitHub REST API

| API | 사용 목적 |
| --- | --- |
| `GET /repos/{owner}/{repo}` | 저장소 정보 |
| `GET /repos/{owner}/{repo}/git/trees/{tree_sha}` | 파일 구조 |
| `GET /repos/{owner}/{repo}/readme` | README |
| `GET /repos/{owner}/{repo}/commits` | Commit 이력 |
| `GET /repos/{owner}/{repo}/contents/{path}` | 파일 내용 |

실제 구현에서는 파일 수와 크기 제한, 페이지네이션, API 호출 제한, 비공개 저장소 접근 권한을 고려해야 합니다.

비공개 저장소 접근이 필요하다면 GitHub App 또는 적절한 OAuth 권한을 사용하고, 인증 토큰은 프론트엔드나 로그에 노출하지 않습니다. 저장소에 포함된 비밀번호·API 키 등의 민감정보는 LLM에 전달하기 전에 탐지·제외해야 합니다.

## 11. Neon PostgreSQL 데이터베이스 설계

| 테이블 | 주요 필드 | 기능 |
| --- | --- | --- |
| `github_repositories` | repository_id, user_id, owner, repo_name, default_branch, github_repo_id | 저장소 등록 |
| `github_analysis_runs` | analysis_id, repository_id, commit_sha, status, started_at, completed_at | 분석 실행 이력 |
| `github_files` | file_id, analysis_id, path, file_type, size, sha | 파일 목록 |
| `github_commits` | commit_id, analysis_id, commit_sha, author, committed_at, message | Commit 이력 |
| `github_analysis_results` | result_id, analysis_id, category, score, summary, findings_json | 분석 결과 |
| `github_analysis_findings` | finding_id, analysis_id, severity, file_path, evidence, recommendation | 개선사항 |
| `task_verifications` | verification_id, task_id, analysis_id, result, evidence_json | 학습과제 검증 |

`github_analysis_runs`에 분석 기준 Commit SHA를 저장하면, 이후 소스코드가 변경되어도 과거 분석 결과가 어떤 코드 상태를 기준으로 생성되었는지 추적할 수 있습니다.

## 12. Backend API 설계

| Method | Endpoint | 기능 |
|---|---|---|
| POST | `/api/github/repositories` | 저장소 등록 |
| GET | `/api/github/repositories` | 사용자 저장소 목록 |
| GET | `/api/github/repositories/{repo_id}` | 저장소 상세 |
| POST | `/api/github/repositories/{repo_id}/analyze` | 분석 실행 |
| GET | `/api/github/analyses/{analysis_id}/status` | 분석 진행 상태 |
| GET | `/api/github/analyses/{analysis_id}` | 분석 결과 조회 |
| GET | `/api/github/analyses/{analysis_id}/files` | 파일 구조 조회 |
| GET | `/api/github/analyses/{analysis_id}/commits` | Commit 분석 |
| GET | `/api/github/analyses/{analysis_id}/findings` | 개선사항 조회 |
| POST | `/api/github/analyses/{analysis_id}/verify-task` | 학습과제 검증 |
| GET | `/api/github/repositories/{repo_id}/history` | 분석 이력 조회 |

## 12.1 분석 요청 예시

```
{
  "repository_id": "repo_001",
  "branch": "main",
  "task_id": "task_008",
  "analysis_options": {
    "structure": true,
    "readme": true,
    "commits": true,
    "project_review": true
  }
}
```

## 12.2 분석 결과 예시

```
{
  "analysis_id": "analysis_001",
  "status": "completed",
  "repository": "example/ai-devcoach",
  "branch": "main",
  "commit_sha": "FULL_COMMIT_SHA",
  "tech_stack": ["Python", "FastAPI", "SQLAlchemy"],
  "summary": {
    "structure": "backend/frontend 분리",
```

```
    "readme": "실행 방법 보완 필요",
    "testing": "테스트 범위 추가 확인 필요"
  },
  "findings": [
    {
      "severity": "medium",
      "category": "documentation",
      "recommendation": "README에 실행 명령어 추가"
    }
  ],
  "task_verification": {
    "task_id": "task_008",
    "status": "needs_review"
  }
}
```

## 13. 예외 및 오류 처리

| 오류 상황 | 화면 메시지 | 처리 |
|---|---|---|
| URL 형식 오류 | GitHub 저장소 주소를 확인하세요 | 재입력 |
| 저장소 없음 | 저장소를 찾을 수 없습니다 | 주소·권한 확인 |
| 비공개 저장소 | 접근 권한이 필요합니다 | 인증 안내 |
| Branch 없음 | 선택한 Branch가 없습니다 | Branch 재선택 |
| API 호출 제한 | 잠시 후 다시 시도하세요 | 대기·재시도 |
| 파일 과다 | 분석 범위를 제한합니다 | 파일 선별 |
| README 없음 | README를 찾을 수 없습니다 | 문서 작성 제안 |
| LLM 분석 실패 | 분석을 완료하지 못했습니다 | 재시도 |
| 근거 부족 | 구현 여부를 확인할 수 없습니다 | 추가 증빙 요청 |
| 분석 중 중복 요청 | 동일 분석이 진행 중입니다 | 기존 작업 상태 표시 |

## 14. 테스트 케이스

| TC ID | 테스트 항목 | 기대 결과 |
|---|---|---|
| GH-T01 | 정상 저장소 URL 입력 | 저장소 등록 |
| GH-T02 | 잘못된 URL 입력 | 입력 오류 |
| GH-T03 | 존재하지 않는 저장소 | 오류 안내 |
| GH-T04 | 비공개 저장소 접근 | 인증·권한 확인 |

| TC ID | 테스트 항목 | 기대 결과 |
|---|---|---|
| GH-T05 | Branch 선택 | 해당 Branch 분석 |
| GH-T06 | 파일 구조 조회 | 파일 목록 표시 |
| GH-T07 | README 분석 | 기준별 평가 결과 |
| GH-T08 | Commit 조회 | 지정 기간 데이터 표시 |
| GH-T09 | AI 프로젝트 분석 | 근거 있는 결과 생성 |
| GH-T10 | 분석 재실행 | 새 분석 이력 생성 |
| GH-T11 | 학습과제 연계 | 완료 기준과 코드 비교 |
| GH-T12 | 검증 근거 부족 | 확인 불가 처리 |
| GH-T13 | 타 사용자 저장소 접근 | 권한 거부 |
| GH-T14 | 민감정보 포함 코드 | LLM 전달 차단·제외 |
| GH-T15 | GitHub API 제한 | 오류 처리 및 재시도 |
| GH-T16 | 분석 결과 저장 | Neon DB 정상 저장 |

## 15. 개발 일정 및 역할 분담

3명 팀의 AI DevCoach 최종 프로젝트를 기준으로 한 목표 공수입니다.

| 작업 | 담당 | 목표 공수 |
|---|---|---|
| GitHub 등록·대시보드 UI | Frontend | 5h |
| 파일 구조·README·Commit UI | Frontend | 5h |
| GitHub API 연동 | Backend | 6h |
| Neon DB 모델·분석 이력 API | Backend | 5h |
| Project Analysis Agent | AI Agent | 7h |
| 학습과제 연계 평가 | AI Agent + Backend | 4h |
| 테스트 및 통합 | 공동 | 4h |
| 합계 | | 36h |

이는 기존 240시간 프로젝트 안에서 배정할 목표 작업량이며, FR-06의 파일별 심층 코드 리뷰 개발은 별도 범위로 둡니다.

## 16. MVP 개발 범위와 최종 완료 기준

240시간 프로젝트에서 FR-05의 범위가 지나치게 커지지 않도록 1차 구현은 공개 GitHub 저장소 분석으로 제한하는 것이 적절합니다.

### 1차 필수 구현

- GitHub 저장소 URL 등록 및 접근 확인
- 기본 Branch와 파일 구조 조회
- README 및 기술 스택 분석
- 최근 Commit 활동 조회
- LLM 기반 프로젝트 구조·문서·구현 현황 분석
- 분석 결과 및 근거를 Neon PostgreSQL에 저장
- 학습과제 완료 기준과 GitHub 결과물 비교
- 분석 결과 대시보드 표시 및 재분석

### 2차 확장 기능

- GitHub OAuth 또는 GitHub App을 통한 비공개 저장소 분석
- Pull Request 및 협업 활동 평가
- 코드 정적 분석 도구 연계
- Commit 간 변경점 기반 AI 리뷰
- 프로젝트 실행·자동 테스트
- 분석 결과에 따른 학습계획 자동 재추천

FR-05의 핵심 완료 기준은 학습자가 GitHub URL을 입력하면 실제 프로젝트 데이터를 수집하고, AI가 분석 근거와 개선사항을 제시하며, 그 결과를 학습과제 평가에 활용할 수 있는 것입니다.

이를 통해 AI DevCoach는 단순히 학습 진도를 기록하는 시스템을 넘어, 학습자가 실제로 작성한 코드와 프로젝트 결과물을 바탕으로 역량 향상을 지원하는 AI 학습 코치로 발전할 수 있습니다.

---

## OpenAI API 활용범위

AI DevCoach의 GitHub 프로젝트 분석 기능(FR-05)에서는 OpenAI API를 사용하는 것을 권장합니다. 다만 GitHub 데이터 수집부터 모든 분석을 OpenAI API로 처리하는 것은 아닙니다.

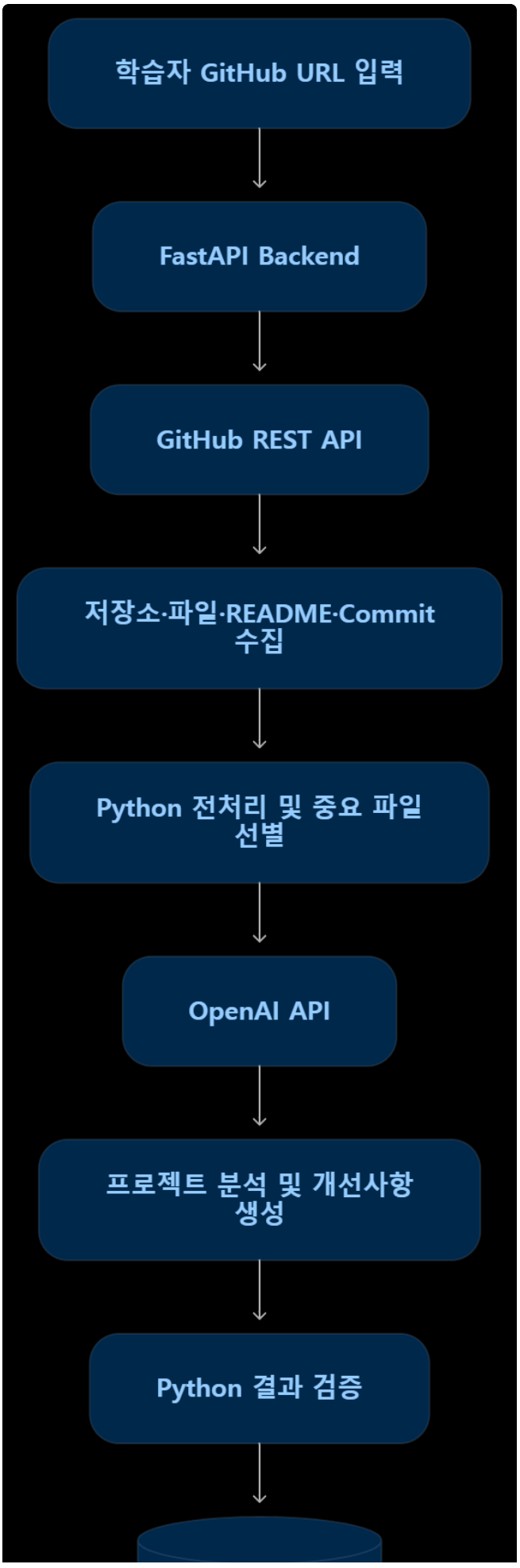
GitHub API는 실제 코드와 프로젝트 정보를 수집하고, OpenAI API는 수집한 내용을 이해·평가하여 개선사항을 제안하는 역할을 담당합니다.

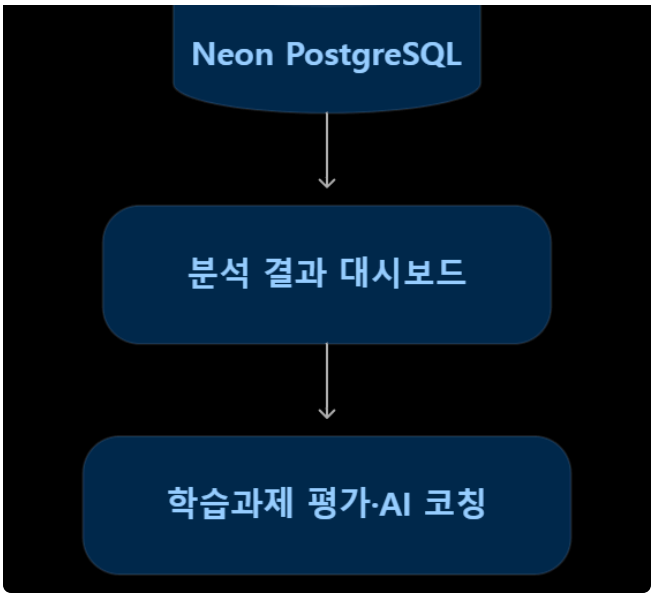
## 1. GitHub API와 OpenAI API 역할 구분

| 분석 기능 | GitHub API | OpenAI API |
|---|---|---|
| 저장소 이름·설명 조회 | 사용 | 불필요 |
| 폴더·파일 목록 수집 | 사용 | 불필요 |

| 분석 기능 | GitHub API | OpenAI API |
|---|---|---|
| README 내용 수집 | 사용 | 내용 평가 |
| Commit 이력 조회 | 사용 | 변경 내용 해석 |
| 기술 스택 확인 | 파일·설정 수집 | 필요시 보조 분석 |
| 프로젝트 구조 평가 | 근거 데이터 제공 | 사용 |
| 코드 품질 분석 | 소스코드 수집 | 사용 |
| 학습과제 달성 평가 | 구현 증거 수집 | 사용 |
| 개선사항·학습 추천 | 불필요 | 사용 |

## 2. AI DevCoach 분석 아키텍처


학습자 GitHub URL 입력
FastAPI Backend
GitHub REST API
저장소·파일·README·Commit 수집
Python 전처리 및 중요 파일 선별
OpenAI API
프로젝트 분석 및 개선사항 생성
Python 결과 검증



전처리 및 중요 파일선별OpenAI API프로젝트 분석 및 개선사항생성Python 결과 검증Neon PostgreSQL분석 결과 대시보드학습과제 평가·AI 코칭

## 3. OpenAI API가 실제로 수행하는 분석

예를 들어 학습자가 FastAPI 프로젝트를 GitHub에 등록했다고 가정하겠습니다.

GitHub API에서 `main.py`, `database.py`, `models.py`, `README.md`를 수집한 후 OpenAI API에 다음 작업을 요청합니다.

1. 프로젝트의 주요 기능과 기술 스택을 설명한다.
2. 코드 구조가 적절하게 분리되었는지 평가한다.
3. 오류 처리, 데이터 검증, 보안상 개선사항을 찾는다.
4. README에 부족한 내용을 확인한다.
5. 학습자의 과제 완료 기준과 실제 구현 코드를 비교한다.
6. 우선적으로 수정해야 할 사항 3개를 제시한다.

## Python 코드 구현 예시

```
from openai import OpenAIclient = OpenAI()def analyze_github_project(readme, source_code):    response = client.responses.create(        model="gpt-5.4-mini",        instructions=(            "당신은 AI 개발 프로젝트 평가자입니다. "
"제공된 코드와 README만 근거로 분석하세요. "            "확인되지 않은 기능은 구현되었다고 판단하지 마세요."        ),        input=f"""        [README]
{readme}        [SOURCE CODE]        {source_code}        다음 항목을 분석하세요.
1. 프로젝트 주요 기능        2. 기술 스택        3. 코드 구조의 적절성        4. 발견된 문제점        5. 개선 우선순위 3개        """    )    return response.output_text
```

위 코드는 OpenAI API 호출 부분만 보여주는 최소 예시입니다. 실제 서비스에서는 파일 크기 제한, 민감정보 제거, 오류 처리, JSON 구조화 출력, 분석 근거 검증이 추가되어야 합니다. 모델 사용 가능 여부도 API 계정에서 확인해야 합니다.

## 4. OpenAI API 없이 가능한 분석

OpenAI API를 사용하지 않아도 Python으로 다음 항목을 분석할 수 있습니다.

- 파일·폴더 개수 및 구성
- 사용 언어와 프레임워크 탐지
- README 존재 여부
- Commit 수 및 주간 활동량
- `requirements.txt` 분석
- Python AST 기반 함수·클래스 구조 추출
- Ruff 등의 도구를 활용한 정적 코드 검사

따라서 Python 기반의 정량·규칙 분석 + OpenAI 기반의 정성·의미 분석을 결합하는 것이 비용과 분석 신뢰성 측면에서 유리합니다.

## 5. AI DevCoach 최종 프로젝트 권장 구현 범위

| 구현 단계 | 기술 | 결과물 |
|---|---|---|
| 1단계 | GitHub REST API | 저장소·README·소스코드 수집 |
| 2단계 | Python | 파일 구조 및 기술 스택 분석 |
| 3단계 | OpenAI API | 프로젝트 평가 및 개선사항 |
| 4단계 | Python | 분석 근거·결과 검증 |
| 5단계 | Neon PostgreSQL | 분석 결과와 이력 저장 |
| 6단계 | FastAPI + HTML/JS | 분석 대시보드 출력 |

권장사항: 1차 MVP에서는 README와 핵심 소스코드 3~5개만 OpenAI API에 전달하여 프로젝트 분석을 구현하세요. 전체 저장소를 매번 분석하면 토큰 비용과 응답시간이 증가합니다.

이후 FR-06 AI 코드 리뷰에서는 동일한 OpenAI API를 재사용하되, 프로젝트 전체 구조 분석보다 개별 코드의 오류·개선사항·수정 예제 생성에 집중하도록 기능을 분리하는 것이 좋습니다.